

Laboratorio I

a.a. 2025/2026

TypeScript III

Generics

In svariati linguaggi, uno degli strumenti principali per scrivere componenti riusabili sono i **generics**:
permettono di creare componenti che possono funzionare con una varietà di tipi piuttosto che uno solo



Generics

Iniziamo dall' "Hello World" dei **generics**: la funzione identità!

```
function identity(arg: number): number {  
  return arg;  
}
```

Senza **generics**, per poter utilizzare la funzione identità con altri tipi, dovremmo scriverne una per ogni possibile tipo, oppure come potremmo fare usando un tipo TS già visto?

Generics

Iniziamo dall' "Hello World" dei **generics**: la funzione identità!

```
function identity(arg: number): number {  
  return arg;  
}
```

Senza **generics**, per poter utilizzare la funzione identità con altri tipi, dovremmo scriverne una per ogni possibile tipo, oppure come potremmo fare usando un tipo TS già visto?

```
function identity(arg: any): any {  
  return arg;  
}
```

In questo, quando ritorniamo, stiamo **perdendo informazione** sul tipo passato. Ad esempio, se passiamo un `number`, l'unica info che abbiamo è che qualsiasi (`any`) tipo viene restituito

Generics

Abbiamo bisogno di un modo per **catturare il tipo** dell'argomento, così da poterlo usare per denotare il tipo che viene restituito

Usiamo una **type variable**, una speciale variabile che funziona con tipi piuttosto che con valori



Generics

Abbiamo bisogno di un modo per **catturare il tipo** dell'argomento, così da poterlo usare per denotare il tipo che viene restituito

Usiamo una **type variable**, una speciale variabile che funziona con tipi piuttosto che con valori

```
function identity<T>(arg: T): T {  
  return arg;  
}
```

Abbiamo usato una **type variable T**, che permette di *catturare* il tipo del dato passato dal chiamante (ad es., **number**), per poterlo poi usare dopo (as es., come *tipo di ritorno* della funzione)



Generics

```
function identity<T>(arg: T): T {  
  return arg;  
}
```

Questa versione della funzione è **generica**, dato che funziona con *diversi tipi*

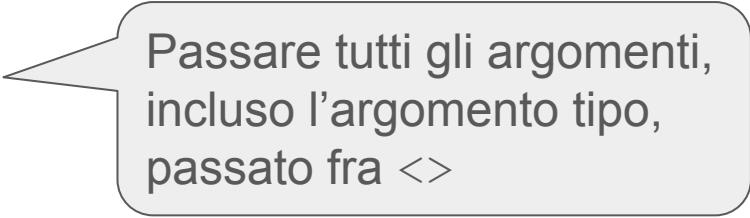
A differenza di **any**, è *precisa* (non perde informazione) tanto quanto la prima versione che usava *direttamente il tipo number*

Generics

```
function identity<T>(arg: T): T {  
  return arg;  
}
```

Una volta definita, la funzione generica si può invocare in due modi

```
let output = identity<string>("myString");  
//      ^ = let output: string
```



Passare tutti gli argomenti,
incluso l'argomento tipo,
passato fra <>

Generics

```
function identity<T>(arg: T): T {  
  return arg;  
}
```

Una volta definita, la funzione generica si può invocare in due modi

```
let output = identity<string>("myString");  
//      ^ = let output: string
```

```
let output = identity("myString");  
//      ^ = let output: string
```

type argument inference: il compilatore imposta il valore per `T` automaticamente, basandosi sul tipo passato come argomento (non sempre possibile)

Generics

- Generics però non vuol dire che possiamo fare quello che ci pare con <T>

```
function loggingIdentity<T>(arg: T): T {  
  console.log(arg.length); // KO Property 'length' does not exist on type 'T'  
  return arg;  
}
```

length non è
definito per tutti i
tipi!

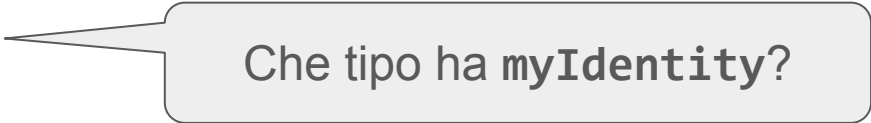


Generics types

```
function identity<T>(arg: T): T {  
  return arg;  
}
```

E diciamolo di nuovo: le funzioni hanno un loro tipo, eventualmente “generico”

```
let myIdentity = identity;
```



Che tipo ha `myIdentity`?

Generics types

```
function identity<T>(arg: T): T {  
  return arg;  
}
```

E diciamolo di nuovo: le funzioni hanno un loro tipo, eventualmente “generico”

```
let myIdentity = identity;
```

```
let myIdentity: <U>(arg: U) => U = identity;
```

Che possiamo inglobare in
una interfaccia!

```
interface GenericIdentityFn {  
  <T>(arg: T): T;  
}
```

Generics types

```
function identity<T>(arg: T): T {  
  return arg;  
}
```

E diciamolo di nuovo: le funzioni hanno un loro tipo, eventualmente “generico”

```
let myIdentity: <U>(arg: U) => U = identity;
```

```
interface GenericIdentityFn {  
  <T>(arg: T): T;  
}
```

```
let myIdentity: GenericIdentityFn = identity;
```

Generics types

```
function identity<T>(arg: T): T {  
  return arg;  
}
```

E diciamolo di nuovo: le funzioni hanno un loro tipo, eventualmente “generico”

```
let myIdentity: <U>(arg: U) => U = identity;
```

```
interface GenericIdentityFn {  
  <T>(arg: T): T;  
}
```

```
let myIdentity: GenericIdentityFn = identity;
```

Abbiamo già visto le interfacce per descrivere invece la “forma” di oggetti

```
interface Coordinate {  
  x: number;  
  y: number;  
}  
function printXY(o: Coordinate) {  
  console.log(`(${o.x},${o.y})`)  
}  
let o = {x: 20, y: 100, z: 200};  
printXY(o)
```

Generics types

```
interface GenericIdentityFn {  
  <T>(arg: T): T;  
}
```

Per rendere ancora più generica l'interfaccia rispetto al type variable (così da renderla visibile anche ai membri dell'interfaccia):

Generics types

```
interface GenericIdentityFn {  
  <T>(arg: T): T;  
}
```

Per rendere ancora più generica l'interfaccia rispetto al type variable (così da renderla visibile anche ai membri dell'interfaccia):

```
interface GenericIdentityFn<T> {  
  (arg: T): T;  
}
```

```
function identity<T>(arg: T): T {  
  return arg;  
}
```

```
let myIdentity: GenericIdentityFn<number> = identity;
```

Interfacce generiche

L'idea dei generics chiaramente non è limitata alle sole funzioni!

```
interface Nodo {  
  value: number;  
  next: Nodo | null;  
  prev: Nodo | null;  
}
```

vs

Generic interface

```
interface Nodo<T> {  
  value: T;  
  next: Nodo<T> | null;  
  prev: Nodo<T> | null;  
}
```

Interfacce generiche

L'idea dei generics chiaramente non è limitata alle sole funzioni!

```
interface Nodo {  
  value: number;  
  next: Nodo | null;  
  prev: Nodo | null;  
}
```

vs

Generic interface

```
interface Nodo<T> {  
  value: T;  
  next: Nodo<T> | null;  
  prev: Nodo<T> | null;  
}
```

Generic interface usata come tipo

```
let n1: Nodo<number> = {value: 1, next: null, prev: null};
```

Classi generiche

```
class GenericValue<T> {  
    value: T;  
    constructor(value: T) {  
        this.value = value  
    }  
    equals(y : T) : boolean {  
        return this.value == y  
    }  
}  
  
let s = new GenericValue<string>("bla")  
console.log(s.equals("bla"))  
  
let i = new GenericValue<boolean>(true)  
console.log(i.equals(false))
```

```
class Pair<T, U> {  
    #p : [T, U]  
    constructor(first: T, second: U) {  
        this.#p = [first, second]  
    }  
    get first() : T { return this.#p[0] }  
    get second() : U { return this.#p[1] }  
    get getPair(): [T,U] {  
        return [this.first, this.second]  
    }  
}  
  
let c = new Pair<string, number>("abc", 1)  
console.log(c.second)
```

Interfacce vs classi generiche

```
interface PizzaI<T> {  
    name: string;  
    toppings: T[];  
}
```

VS

```
class Pizza<T> {  
    name: string;  
    toppings: T[];  
}
```

Interfacce vs classi generiche

```
interface PizzaI<T> {  
  name: string;  
  toppings: T[];  
}
```

VS

```
class Pizza<T> {  
  name: string;  
  toppings: T[];  
}
```

Interfaccia non definisce
implementazione di metodi
(incluso il costruttore)

Interfaccia usata in TS solo per
scopi legati al type check (dopo il
transpiling, non esiste più)

Da una classe potete creare
oggetti, e la struttura che
definite continua ad esistere in
JS (dopo il *transpiling*)

Interfacce vs classi generiche

TS

```
interface PizzaI<T> {  
  name: string;  
  toppings: T[];  
}
```

```
let p : PizzaI<string> = {name: "rossa",  
  toppings: ["pomodoro"]}
```

TS

```
class Pizza<T> {  
  name: string;  
  toppings: T[];  
  constructor(n: string, t: T[]) {  
    this.name = n; this.toppings = t  
  }  
}
```

```
let p = new Pizza<string>("rossa", ["pomodoro"])
```

JS

```
"use strict";  
let p = { name: "rossa", toppings: ["pomodoro"] };
```

JS

```
"use strict";  
class Pizza {  
  constructor(n, t) {  
    this.name = n;  
    this.toppings = t;  
  }  
}  
let p = new Pizza("rossa", ["pomodoro"]);
```

Le interfacce, come le classi, si possono estendere, es.

`interface GourmetPizza<T> extends PizzaI<T> { ... }`

```
interface PizzaI<T> {  
  name: string;  
  toppings: T[];  
}
```

TS

```
let p : PizzaI<string> = {name: "rossa",  
  toppings: ["pomodoro"]}
```

"use strict";

```
let p = { name: "rossa", toppings: ["pomodoro"] };
```

JS

Le classi possono implementare una o più interfacce, es.

`class Margherita<T> implements PizzaI<T> { ... }`

```
class Pizza<T> {  
  name: string;  
  toppings: T[];  
  constructor(n: string, t: T[]) {  
    this.name = n; this.toppings = t  
  }  
}
```

TS

```
let p = new Pizza<string>("rossa", ["pomodoro"])
```

"use strict";

```
class Pizza {  
  constructor(n, t) {  
    this.name = n;  
    this.toppings = t;  
  }  
}
```

JS

```
let p = new Pizza("rossa", ["pomodoro"]);
```

Generics constraints



- Riprendiamo l'esempio in cui non possiamo fare quello che vogliamo!

```
function loggingIdentity<T>(arg: T): T {  
  console.log(arg.length); // KO: Property 'length' does not exist on type 'T'  
  return arg;  
}
```

- Invece di lavorare con tutti i tipi `<T>`, vogliamo limitarci solo a quelli che hanno la proprietà **length** disponibile
 - Dobbiamo quindi introdurre una limitazione (*constraint*) su cosa `T` può essere

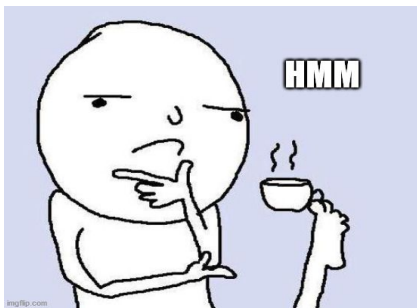
Generics constraints



- Riprendiamo l'esempio in cui non possiamo fare quello che vogliamo!

```
function loggingIdentity<T>(arg: T): T {  
  console.log(arg.length); // KO: Property 'length' does not exist on type 'T'  
  return arg;  
}
```

- Invece di lavorare con tutti i tipi $<T>$, vogliamo limitarci solo a quelli che hanno la proprietà **length** disponibile
 - Dobbiamo quindi introdurre una limitazione (*constraint*) su cosa T può essere



Idee?

Ingredienti

- Interfacce, extends e implements
- Generics
- Proprietà
- E tutto quello che conoscete

Generics constraints

```
interface Lengthwise {  
  length: number;  
}
```

```
function loggingIdentity<T extends Lengthwise>(arg: T): T {  
  console.log(arg.length); // Ora <T> ha la proprietà .length  
  return arg;  
}
```

```
loggingIdentity({ length: 10, value: 3 });
```

Classi: visibilità di metodi e proprietà

TS introduce `public`, `protected` e `private`

```
class Greeter {  
  private a: number = 0;  
  public greet() {  
    console.log("Hello, " + this.getName());  
  }  
  protected getName() {  
    return "hi";  
  }  
}
```

- `public`: visibilità di default
- `protected`: visibile alla classe o alle sottoclassi
- `private`: visibile solo alla classe

Classi: visibilità di metodi e proprietà

TS

```
1 class Greeter {  
2   private a: number = 0;  
3   public greet() {  
4     console.log("Hello, " + this.getName());  
5   }  
6   protected getName() {  
7     return "hi";  
8   }  
9 }
```

JS

```
"use strict";  
class Greeter {  
  constructor() {  
    this.a = 0;  
  }  
  greet() {  
    console.log("Hello, " + this.getName());  
  }  
  getName() {  
    return "hi";  
  }  
}
```

Classi: visibilità di metodi e proprietà

TS

```
1 class Greeter {  
2   private a: number = 0;  
3   public greet() {  
4     console.log("Hello, " + this.getName());  
5   }  
6   protected getName() {  
7     return "hi";  
8   }  
9 }
```

JS

```
"use strict";  
class Greeter {  
  constructor() {  
    this.a = 0;  
  }  
  greet() {  
    console.log("Hello, " + this.getName());  
  }  
  getName() {  
    return "hi";  
  }  
}
```

I modificatori di visibilità in TS **non esistono a runtime**: dopo il *transpiling* sono proprietà accessibili come tutte le altre!

Classi: visibilità di metodi e proprietà

TS

```
1 class Pizza {  
2   constructor(public name: string, public toppings: string[]) {  
3   };  
4 }  
5  
6  
7  
8
```

JS

```
"use strict";  
class Pizza {  
  constructor(name, toppings) {  
    this.name = name;  
    this.toppings = toppings;  
  }  
};
```

Uso "strano"

Q & A